1. Introduction of Operating System

a. Functions of operating system
b. Types of operating system
c. History of operating system
d. Structure of operating system

OBJECTIVES:

- To learn the fundamentals of Operating Systems.
- To learn the mechanisms of OS to handle processes and threads and their communication
- To learn the mechanisms involved in memory management in contemporary OS
- To gain knowledge on distributed operating system concepts that includes architecture
- deadlock detection algorithms and agreement protocols
- To know the components and management aspects of concurrency management

---

A program that acts as an intermediary between a user of a computer and the computer hardware.

An operating System is a collection of system programs that together control the operations of a computer system.

Some examples of operating systems are UNIX, MS-DOS, MS-Windows, Windows/NT, Chicago, OS/2, MacOS, and VM.

Operating system goals:

Execute user programs and make solving user problems easier

.• Make the computer system convenient to use.

- Use the computer hardware in an efficient manner.

- Computer System Components

a. Functions of operating system

Its primary purpose is to provide a platform for executing programs efficiently. Its main functions include:

1. **Process Management**:
    - **Creating and Terminating Processes**: The OS is responsible for creating and terminating processes as needed.
    - **Scheduling**: It schedules processes for execution, ensuring fair allocation of CPU time among processes.

- **Synchronization and Communication**: It manages process synchronization and communication through mechanisms like message queues, and shared memory.

2. **Memory Management**:
   - **Allocation and Deallocation**: The OS allocates and deallocates memory spaces as required by processes.
   - **Paging and Segmentation**: It uses techniques like paging and segmentation to manage memory efficiently and provide protection.
   - **Virtual Memory**: The OS enables virtual memory, allowing processes to use more memory than physically available by swapping data between RAM and disk storage.
3. **File System Management**:
   - **File Organization**: The OS manages how data is stored, organized, and accessed on storage devices.
   - **Directory Management**: It provides a hierarchical structure to organize files into directories.
   - **Security and Access Control**: The OS ensures security and access control, restricting access to files based on user permissions.
4. **Device Management**:
   - **Device Drivers**: The OS includes device drivers that act as translators between the hardware devices and the applications or system.
   - **I/O Management**: It manages input/output operations, buffering, caching, and spooling.
   - **Resource Allocation**: The OS allocates and deallocates devices efficiently among competing processes.
5. **Security and Protection**:
   - **User Authentication**: The OS handles user authentication to ensure that only authorized users can access the system.
   - **Access Control**: It manages access control policies, ensuring that users and processes have the appropriate permissions.
   - **Data Security**: The OS protects data from unauthorized access and ensures data integrity.
6. **User Interface**:
   - **Command-Line Interface (CLI)**: Provides a text-based interface where users can type commands to interact with the system.
   - **Graphical User Interface (GUI)**: Offers a visual interface with graphical elements like windows, icons, and menus for easier interaction.
7. **System Performance Monitoring**:
   - **Performance Monitoring**: The OS monitors system performance, tracking CPU usage, memory usage, and other critical metrics.
   - **Logging and Reporting**: It maintains logs of system activities and errors, providing reports for troubleshooting and optimization.
8. **Networking**:
   - **Network Communication**: The OS facilitates network communication by implementing protocols and managing network connections.
   - **Resource Sharing**: It allows resources such as files, printers, and internet connections to be shared across a network.

b. Types of operating system

Operating systems can be categorized based on various way such as their capabilities, the devices they manage, and their intended use. Operating systems play a vital role in managing computer resources and enabling efficient execution of programs.

Here are the main types of operating systems:

1. **Batch Operating Systems**:

   - Doesn't interact directly with the computer.
   - Jobs with similar requirements are grouped into batches.
   - Examples: Payroll systems, bank statements.

     - **Characteristics**: Execute batches of jobs without user interaction. Jobs are collected, processed, and executed in batches.
     - **Use Case**: Mainly used in early computers where users did not interact directly with the machine.
2. **Time-Sharing Operating Systems**:

   - Shares CPU time among multiple users or processes.
   - Enables interactive computing and responsiveness.

     - **Characteristics**: Allow multiple users to share system resources simultaneously by providing each user a time slice.
     - **Use Case**: Used in multi-user environments where multiple users need to interact with the system at the same time.
3. **Distributed Operating Systems**:

   - Spans multiple interconnected computers.
   - Supports distributed computing and resource sharing.

     - **Characteristics**: Manage a group of independent computers and make them appear to be a single coherent system.
     - **Use Case**: Used in environments where tasks are distributed across multiple machines for efficiency and reliability.
4. **Network Operating Systems**:

   - Manages network resources and communication.
   - Used in servers and network devices.

     - **Characteristics**: Provide features for networking, including file sharing, hardware sharing, and communication among computers on a network.
     - **Use Case**: Used in network environments where resources and data need to be shared across multiple computers.
5. **Real-Time Operating Systems (RTOS)**:

- Ensures timely execution of critical tasks.
- Used in robotics, aerospace, and industrial control systems.

    - **Characteristics**: Designed to process data as it comes in, typically within a strict time constraint.
    - **Use Case**: Used in systems that require precise timing and reliability, such as embedded systems, medical devices, and industrial control systems.

6. **Embedded Operating Systems**:
    - **Characteristics**: Optimized to run on small devices with limited resources and specific functionalities.
    - **Use Case**: Found in embedded systems like appliances, automotive systems, and dedicated consumer electronics.
7. **Mobile Operating Systems**:
    - **Characteristics**: Designed specifically for mobile devices such as smartphones and tablets, optimized for touch input and mobile-specific hardware.
    - **Use Case**: Examples include Android, iOS, and Windows Phone.
8. **Desktop Operating Systems**:
    - **Characteristics**: Designed for personal computers with a focus on user-friendly interfaces and a broad range of software applications.
    - **Use Case**: Common desktop OSs include Microsoft Windows, macOS, and Linux distributions.
9. **Server Operating Systems**:
    - **Characteristics**: Tailored for server environments, providing robust networking, security, and resource management features.
    - **Use Case**: Examples include Windows Server, Linux server distributions (like Ubuntu Server, CentOS), and UNIX-based systems.
10. **Mainframe Operating Systems**:
    - **Characteristics**: Designed for large-scale computers that manage extensive and complex applications, supporting high-volume transaction processing.
    - **Use Case**: Used in large enterprises for critical applications such as bulk data processing and large-scale transaction processing. Examples include IBM z/OS.
11. **Single-user vs. Multi-user Operating Systems**:
    - **Single-user**: Support one user at a time. Examples include most home and office desktop OSs.
    - **Multi-user**: Allow multiple users to use the system resources simultaneously. Examples include UNIX and mainframe OSs.
12. **Single-tasking vs. Multi-tasking Operating Systems**:
    - **Single-tasking**: Can execute only one task at a time.
    - **Multi-tasking**: Can execute multiple tasks simultaneously. Most modern OSs are multi-tasking, such as Windows, macOS, and Linux.

These types of operating systems cater to different environments and use cases, ensuring that the right OS can be chosen based on the specific needs of the hardware and applications.

c. History of operating system

The history of operating systems (OS) reflects the evolution of computing technology from the early days of computers to the sophisticated systems we use today. Here's an overview of the major milestones in OS development:

1. Early Systems (1940s - 1950s)

- **First Generation (1940s)**: Computers were operated manually, with no operating systems. Programs were written in machine language, and human operators managed all tasks.
- **Second Generation (1950s)**: Batch processing systems were introduced. Programs were submitted in batches, and the computer executed them sequentially without user interaction. Examples include the IBM 701 and UNIVAC(Universal Automatic Computer I) systems.

2. Development of Batch Systems (1960s)

- **Batch Processing Evolution**: Batch systems became more sophisticated with job control languages (JCL) and early forms of multitasking. Computers like the IBM 7090 used these systems.
- **Introduction of Time-Sharing**: MIT's Compatible Time-Sharing System (CTSS) in 1961 and the Dartmouth Time-Sharing System (DTSS) in 1964 allowed multiple users to interact with the computer simultaneously, laying the groundwork for modern multi-user operating systems.

3. Minicomputers and Early Multiprocessing (1960s - 1970s)

- **Multics (1965)**: The Multiplexed Information and Computing Service (Multics) project aimed to develop a highly secure and versatile time-sharing operating system. Though complex, it influenced many later systems, including Unix.
- **UNIX (1969)**: Developed at AT&T's Bell Labs by Ken Thompson, Dennis Ritchie, and others, Unix introduced many concepts still in use today, like hierarchical file systems, simple and elegant design, and portability.

4. Personal Computers and Graphical Interfaces (1970s - 1980s)

- **CP/M (1974)**: The Control Program for Microcomputers (CP/M) by Gary Kildall became a popular OS for early personal computers.
- **Apple II and DOS (Late 1970s)**: Apple's Disk Operating System (DOS) for the Apple II and Microsoft's MS-DOS (1981) for IBM PCs became widely used.
- **Graphical User Interfaces (GUIs)**: Xerox PARC's work on graphical user interfaces influenced later systems. Apple's Macintosh (1984) and Microsoft Windows (1985) brought GUIs to the masses.

5. Networked and Distributed Systems (1980s - 1990s)

- **Networking Integration**: Operating systems began integrating networking capabilities, with TCP/IP becoming standard. Unix systems played a significant role in network development.

- **Distributed Systems**: Research in distributed computing led to systems like Amoeba, developed by Andrew Tanenbaum, which provided a model for networked, distributed OS design.

6. Modern Operating Systems (1990s - Present)

- **Windows Evolution**: Microsoft Windows evolved through versions, from Windows 95 with its integrated GUI and DOS base, to Windows NT introducing a robust kernel architecture. Windows XP, 7, 10, and 11 continued to refine user experience and system performance, becoming dominant in the desktop market.
  - **Linux**: Linus Torvalds released the Linux kernel in 1991, which, combined with GNU software, became the GNU/Linux operating system. It gained popularity due to its open-source nature, stability, and flexibility, becoming widely used in servers, desktops, and embedded systems.
  - **macOS**: Apple's macOS, derived from NeXTSTEP (after Apple acquired NeXT), replaced the classic Mac OS in 2001. It offered a Unix-based foundation with a user-friendly GUI, enhancing stability and performance.
  - **Mobile Operating Systems**: The rise of smartphones and tablets led to the development of mobile OSs like iOS (2007) for Apple's iPhone and Android (2008) by Google, which became the most widely used mobile operating system globally.
  - **Virtualization and Cloud**: The 2000s saw a significant rise in virtualization technologies with software like VMware and hypervisors like Xen. Operating systems like VMware ESXi and Microsoft Hyper-V enabled multiple virtual machines to run on a single physical machine. The growth of cloud computing introduced OSs designed for cloud environments, such as Google's Chrome OS and various custom Linux distributions optimized for cloud infrastructures.

Key Innovations and Trends

- **Security**: Enhancements in OS security have been crucial, with features like SELinux (Security-Enhanced Linux), Windows Defender, and macOS's Gatekeeper aiming to protect against malware and unauthorized access.
- **User Interfaces**: Continued improvements in GUIs and user experience, with modern systems offering touch interfaces, voice control, and more intuitive designs.
- **Performance and Scalability**: Advances in hardware and software optimization have led to operating systems that better manage resources, support more concurrent processes, and scale across various devices from smartphones to supercomputers.
- **Open Source Movement**: The success of Linux and other open-source projects like FreeBSD has shown the power of collaborative development, leading to widespread adoption in servers, supercomputers, and many other areas.
- **Integration and Ecosystems**: Modern operating systems are part of broader ecosystems, integrating seamlessly with cloud services, IoT devices, and providing unified experiences across different types of hardware.

Structure Of An Operating System (OS)

The structure of an operating system (OS) refers to the organization of its components and their interactions. There are several common architectural models used to design operating systems, each with its own advantages and trade-offs. Here are the primary structures:

1. Monolithic Architecture

- In a monolithic architecture, the entire OS runs as a single large process in kernel mode. All OS services such as file system management, memory management, process scheduling, and device drivers are contained within this single kernel.
- **Examples**: UNIX, early versions of Linux.
- **Advantages**:
  - High performance due to direct communication between OS components.
  - Easier access to hardware resources.
- **Disadvantages**:
  - Poor modularity and maintainability.
  - A bug in one part of the kernel can crash the entire system.

2. Layered Architecture

- The OS is divided into a series of layers, each built on top of lower layers. Each layer only interacts with the layer directly below it and provides services to the layer above.
- **Examples**: THE operating system, early versions of Windows NT.
- **Advantages**:
  - Clear modularity, making it easier to debug and maintain.
  - Each layer can be developed and updated independently.
- **Disadvantages**:
  - Potentially less efficient due to the overhead of layer-to-layer communication.
  - Designing appropriate layers can be complex.

3. Microkernel Architecture

- The microkernel architecture minimizes the kernel to include only the most essential functions such as low-level address space management, thread management, and inter-process communication (IPC). Other services run in user space as separate processes.
- **Examples**: QNX, Minix, modern versions of macOS and iOS (based on the XNU kernel which has microkernel aspects).
- **Advantages**:
  - High modularity and flexibility.
  - Increased reliability and security since most services run in user mode.
  - Easier to extend and port to new architectures.
- **Disadvantages**:
  - Potential performance overhead due to more frequent context switches and user/kernel mode transitions.

4. Modular Architecture

Similar to the microkernel architecture but more flexible. The kernel can dynamically load and unload modules (drivers, file systems, etc.) at runtime. Linux, for example, can be considered modular as it supports loadable kernel modules.

- **Examples**: Modern Linux, Solaris.
- **Advantages**:
  - Flexibility in adding or removing functionalities.
  - Maintains some performance benefits of monolithic kernels while providing modularity.
- **Disadvantages**:
  - Complexity in managing dependencies between modules.

5. Hybrid Architecture

- Combines elements of monolithic and microkernel architectures. The kernel is larger than a microkernel but supports modularity. It aims to provide the performance benefits of monolithic kernels while maintaining the modularity of microkernels.
- **Examples**: Windows NT and subsequent versions, macOS.
- **Advantages**:
  - Balanced approach offering both performance and modularity.
  - Easier to maintain and develop than pure monolithic kernels.
- **Disadvantages**:
  - More complex design compared to pure monolithic or microkernel architectures.

6. Client-Server Architecture

- In this architecture, the OS services are implemented as server processes. The client processes request services from these server processes. This model is commonly used in distributed systems.
- **Examples**: Windows NT's user-space subsystems, various network operating systems.
- **Advantages**:
  - Clear separation of services.
  - Facilitates distributed computing.
- **Disadvantages**:
  - Network communication overhead can impact performance.
  - More complex inter-process communication mechanisms needed.

Components of an Operating System

Regardless of the architecture, an OS typically includes the following components:

- **Kernel**: The core part of the OS that manages system resources, including memory, processes, and hardware devices.
- **System Calls and APIs**: Interfaces for applications to request services from the OS.
- **User Interface**: Includes command-line interfaces (CLI) and graphical user interfaces (GUI) that allow users to interact with the system.
- **Device Drivers**: Software modules that control and manage hardware devices.
- **File System**: Manages file operations and storage.
- **Process Management**: Manages processes, including their creation, scheduling, and termination.
- **Memory Management**: Manages system memory, including allocation and deallocation.
- **Security and Access Control**: Ensures the system's security by managing user permissions and protecting resources.